

DLD CEP Report

Design and Implementation of a 5-bit ALU
Section A Electrical Engineering Department

Submitted by:

Maliha Seemi Anwar EE-24004

Sania Tariq EE-24027

Maha Tariq EE-24023

Fareena Khan EE-24002

Submitted to: Sir Fahim

TABLE OF CONTENTS	PAGE NO.
PROBLEM, INTRODUCTION & BACKGROUND	1
ARCHITECTURE , LOGIC & OPERATIONS	2-4
SCHEMATIC , STATE (MACHINE, DIAGRAM , TABLE)	5-6
VERILOG CODE	7-16
CONTROL TABLE & TRUTH TABLE	17
SIMULATION RESULT	17-19
PROBLEM , CONCLUSION & RESULT	20
SUMMARY & CONCLUSION	16

DIGITAL LOGIC DESIGN CEP

PROBLEM:

Problem Activities for Students:

1. Design Specifications:

- Design a 5-bit ALU with register input and output.
- The ALU must support Addition, Subtraction, and Multiplication operations. (Note: Multiplication must be implemented using Adder and shifting operation. No behavior model allowed for Multiplier implementation).
- The input and output port must be registered.

2. Verilog Implementation:

- Write synthesizable Verilog code for your ALU design.
- Implement modules for addition, subtraction, and multiplication operations.
- Include modules for register input and output.

3. Simulation:

- Simulate your Verilog code using a simulation tool (e.g., ModelSim) to ensure correct functionality.
- Provide simulation results and debug any issues that arise.

INTRODUCTION:

The Arithmetic Logic Unit (ALU) plays a critical role in the functioning of a microprocessor, serving as the computational heart of the central processing unit (CPU). In the context of a microprocessor, the ALU is responsible for executing arithmetic and logical operations on data, thereby enabling the processor to perform complex tasks. In this Complex Engineering problem (CEP) you will embark on the exciting journey of designing a 5-bit Arithmetic Logic Unit (ALU) with registered input and output. This CEP is designed to test your skills in digital system design, Verilog programming, simulation, and FPGA implementation.

BACKGROUND:

In digital electronics, the ALU is a combinational circuit that performs arithmetic (like addition and subtraction) and logical operations (such as AND, OR, NOT). It is

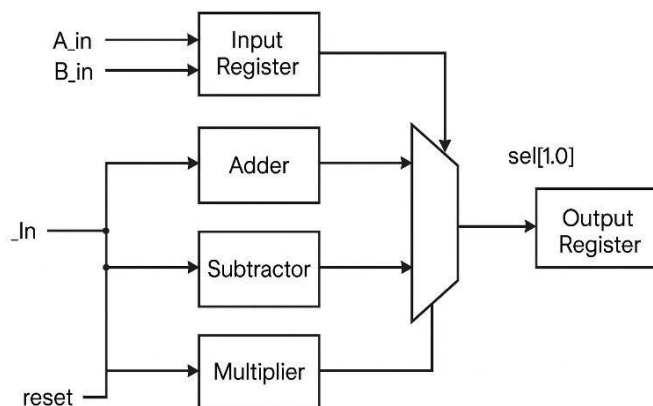
a vital component in microprocessors and digital systems. Traditionally, multiplication operations are complex and are often implemented using behavioral models. However, this project restricts the use of behavioral modeling for multiplication. Instead, multiplication is achieved using repetitive addition and bitwise shifting—demonstrating a deeper understanding of digital system design. This project also incorporates **registers for input and output**, which ensures that the ALU operates synchronously with a clock signal and retains values across cycles.

ARCHITECTURE:

The architecture of the 5-bit ALU consists of the following modules:

- **Input Registers:** Two 5-bit registers store inputs A and B synchronously with the clock.
- **Arithmetic Modules:**
 - **Adder:** A 5-bit adder built using full adder units for bitwise addition.
 - **Subtractor:** Implements subtraction by taking the two's complement of the second operand and adding.
 - **Multiplier:** A 5-bit multiplier using bitwise shifting and addition, unrolled to use gate-level components.
- **Selector (MUX):** A 2-bit selector chooses between addition, subtraction, or multiplication output.
- **Output Register:** A 10-bit register stores the final result.

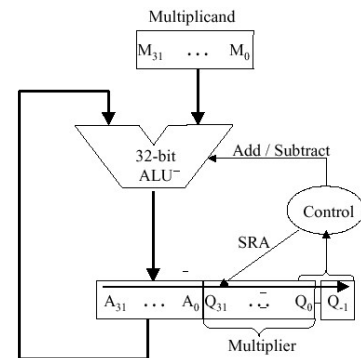
All operations are synchronized to a common clock and controlled via the selector signal.



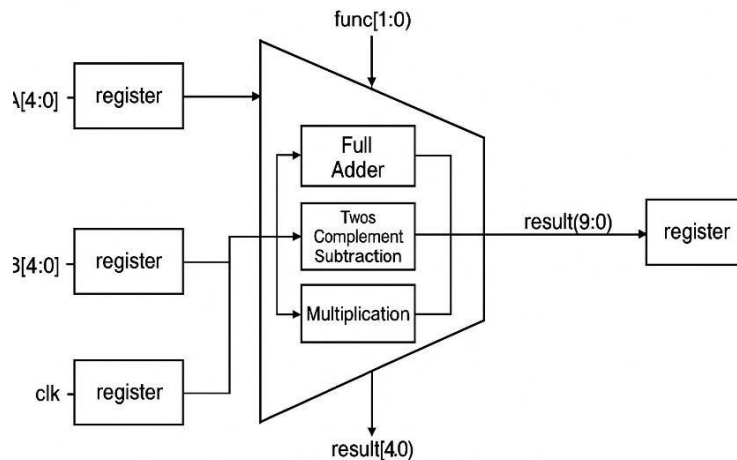
LOGIC:

The ALU logic uses gate-level Verilog (AND, OR, XOR, NOT) to build foundational arithmetic functions.

- **Full Adder Logic:** The adder is implemented using XOR for sum, AND and OR for carry propagation.
- **Two's Complement Subtraction:** The subtractor inverts the input B and adds 1 to compute its two's complement.
- **Multiplication:** Achieved by conditionally shifting and adding the input A for each bit of B. The result is accumulated using chained 10-bit adders.



The 5-bit inputs result in a maximum of 10-bit outputs to support overflow from



multiplication.

OPERATION:

1. **Initialization:**
 - On reset, all registers are cleared.
2. **Input Registration:**
 - Inputs A and B are latched on the rising edge of the clock.

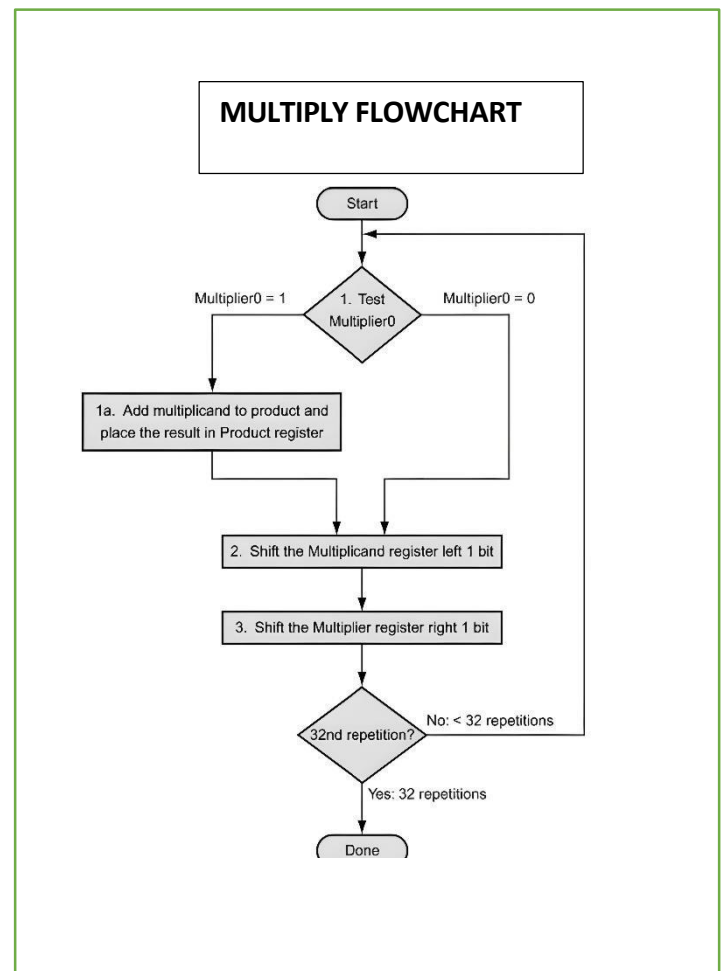
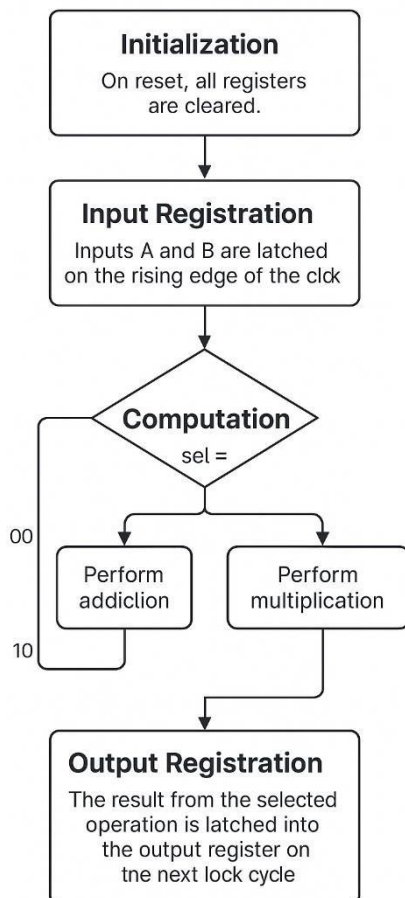
3. Computation:

- Depending on the `sel` input:
 - 00: Perform addition.
 - 01: Perform subtraction.
 - 10: Perform multiplication.

4. Output Registration:

- The result from the selected operation is latched into the output register on the next clock cycle.

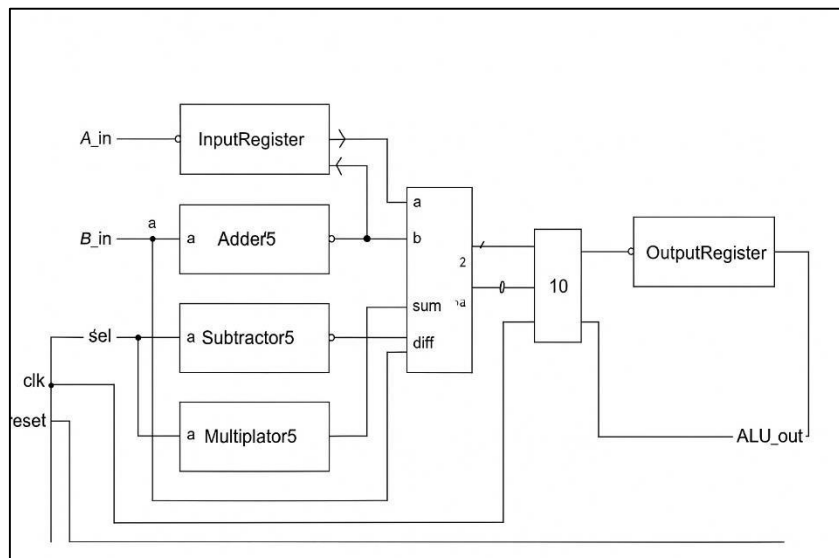
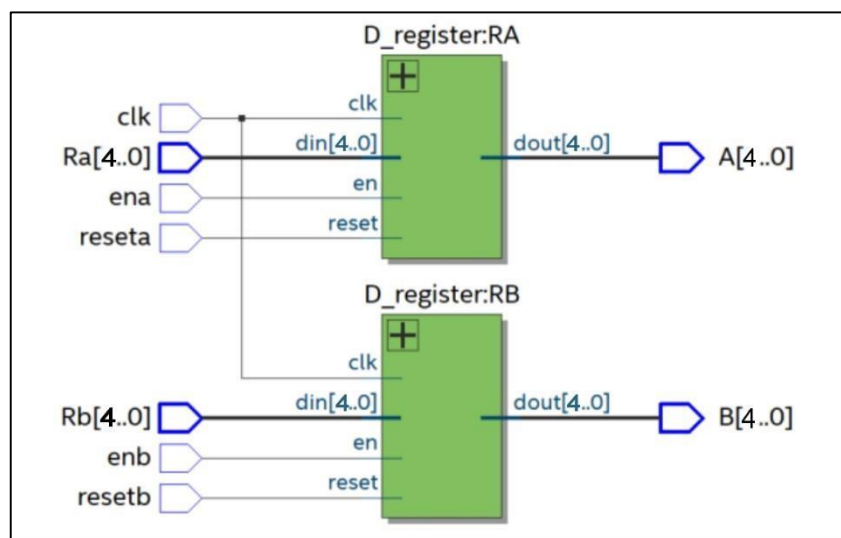
This operation ensures synchronous behavior, allowing seamless integration into larger digital systems such as FPGAs.



SCHEMATIC DIAGRAM:

The ALU consists of the following connected components:

- Two 5-bit input registers (A_reg, B_reg)
- A control selector (sel[1:0]) to choose the operation
- Three processing blocks: 5-bit adder, subtractor, 5x5 multiplier
- A multiplexer to select the operation result
- A 10-bit output register

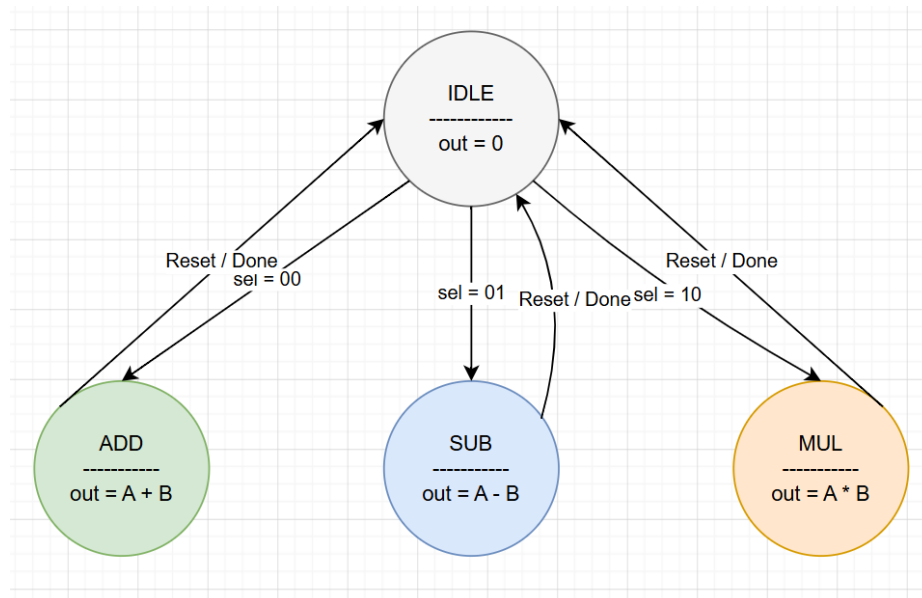


STATE MACHINE USED: Moore FSM

Why Moore?

A Moore machine outputs based solely on the current state. In this ALU, once the inputs are registered and the control input is selected, the ALU transitions to one of the operation states (ADD, SUB, MUL) and produces the output.

STATE DIAGRAM:



STATE TABLE:

Present State	op_code	Next State	Action / Output
ADD_STATE	00	ADD_STATE	result_out = {5'b00000, sum}
SUB_STATE	01	SUB_STATE	result_out = {5'b00000, diff}
MUL_STATE	10	MUL_STATE	result_out = prod
IDLE_STATE	11	IDLE_STATE	result_out = 10'b0

VERILOG CODE:**INPUT REGISTER:**

Ln#	
1	module inputRegister (
2	input clk,
3	input reset,
4	input [4:0] data_in,
5	output reg [4:0] data_out
6	);
7	always @(posedge clk or posedge reset) begin
8	if (reset)
9	data_out <= 5'b0;
10	else
11	data_out <= data_in;
12	end
13	endmodule

TEST BENCH CODE:

1	
2	module tb_inputRegister;
3	reg clk, reset;
4	reg [4:0] data_in;
5	wire [4:0] data_out;
6	
7	InputRegister dut (.clk(clk), .reset(reset), .data_in(data_in), .data_out(data_out));
8	
9	initial begin
10	clk = 0;
11	forever #5 clk = ~clk;
12	end
13	
14	initial begin
15	reset = 1; data_in = 5'b00000; #10;
16	reset = 0; data_in = 5'b10101; #10;
17	data_in = 5'b11100; #10;
18	\$finish;
19	end
20	endmodule

ADDER:

```

1  module FullAdder (
2      input a, b, cin,
3      output sum, cout
4  );
5      wire axorb, aandb, axorbandcin;
6      xor (axorb, a, b);
7      xor (sum, axorb, cin);
8      and (aandb, a, b);
9      and (axorbandcin, axorb, cin);
10     or (cout, aandb, axorbandcin);
11 endmodule
12
13 module adder (
14     input [4:0] a, b,
15     output [4:0] sum
16 );
17     wire [4:0] c;
18     FullAdder fa0 (a[0], b[0], 0, sum[0], c[0]);
19     FullAdder fa1 (a[1], b[1], c[0], sum[1], c[1]);
20     FullAdder fa2 (a[2], b[2], c[1], sum[2], c[2]);
21     FullAdder fa3 (a[3], b[3], c[2], sum[3], c[3]);
22     FullAdder fa4 (a[4], b[4], c[3], sum[4], );
23 endmodule

```

TEST BENCH CODE:

```

27 module tb_adder;
28     reg [4:0] a, b;
29     wire [4:0] sum;
30
31     Adder5 dut (.a(a), .b(b), .sum(sum));
32
33     initial begin
34         a = 5'b00011; b = 5'b00001; #10;
35         a = 5'b01010; b = 5'b00110; #10;
36         a = 5'b11111; b = 5'b00001; #10;
37         $finish;
38     end
39 endmodule
40

```

SUBTRACTOR:

```

1  module subtractor (
2      input [4:0] a, b,
3      output [4:0] diff
4  );
5      wire [4:0] b_comp, one, b_twos;
6
7      assign one = 5'b00001;
8      not (b_comp[0], b[0]);
9      not (b_comp[1], b[1]);
10     not (b_comp[2], b[2]);
11     not (b_comp[3], b[3]);
12     not (b_comp[4], b[4]);
13
14     Adder5 add1 (.a(b_comp), .b(one), .sum(b_twos));
15     Adder5 add2 (.a(a), .b(b_twos), .sum(diff));
16 endmodule

```

TEST BENCH CODE:

```

20 module tb_subtractor;
21     reg [4:0] a, b;
22     wire [4:0] diff;
23
24     Subtractor5 dut (.a(a), .b(b), .diff(diff));
25
26     initial begin
27         a = 5'b10100; b = 5'b00100; #10;
28         a = 5'b01010; b = 5'b01010; #10;
29         a = 5'b00100; b = 5'b01000; #10;
30         $finish;
31     end
32 endmodule

```

MULTIPLIER:

```

Ln#
1  module Adder10 (
2      input [9:0] a, b,
3      output [9:0] sum
4  );
5      wire [9:0] c;
6      FullAdder fa0 (a[0], b[0], 0, sum[0], c[0]);
7      FullAdder fa1 (a[1], b[1], c[0], sum[1], c[1]);
8      FullAdder fa2 (a[2], b[2], c[1], sum[2], c[2]);
9      FullAdder fa3 (a[3], b[3], c[2], sum[3], c[3]);
10     FullAdder fa4 (a[4], b[4], c[3], sum[4], c[4]);
11     FullAdder fa5 (a[5], b[5], c[4], sum[5], c[5]);
12     FullAdder fa6 (a[6], b[6], c[5], sum[6], c[6]);
13     FullAdder fa7 (a[7], b[7], c[6], sum[7], c[7]);
14     FullAdder fa8 (a[8], b[8], c[7], sum[8], c[8]);
15     FullAdder fa9 (a[9], b[9], c[8], sum[9], );
16 endmodule
17
18 module multiplier (
19     input [4:0] a, b,
20     output [9:0] product
21 );
22     wire [9:0] partial0, partial1, partial2, partial3, partial4;
23     wire [9:0] sum1, sum2, sum3, sum4;
24
25     assign partial0 = b[0] ? ({5'b000000, a} : 10'b0;
26     assign partial1 = b[1] ? ({5'b000000, a} << 1) : 10'b0;
27     assign partial2 = b[2] ? ({5'b000000, a} << 2) : 10'b0;
28     assign partial3 = b[3] ? ({5'b000000, a} << 3) : 10'b0;
29     assign partial4 = b[4] ? ({5'b000000, a} << 4) : 10'b0;
30
31     Adder10 add1 (.a(partial0), .b(partial1), .sum(sum1));
32     Adder10 add2 (.a(sum1), .b(partial2), .sum(sum2));
33     Adder10 add3 (.a(sum2), .b(partial3), .sum(sum3));
34     Adder10 add4 (.a(sum3), .b(partial4), .sum(product));
35 endmodule

```

TEST BENCH CODE:

```

37
38 module tb_multiplier;
39     reg [4:0] a, b;
40     wire [9:0] product;
41
42     Multiplier5 dut (.a(a), .b(b), .product(product));
43
44     initial begin
45         a = 5'b00011; b = 5'b00010; #10; // 3 * 2
46         a = 5'b00101; b = 5'b00100; #10; // 5 * 4
47         a = 5'b11111; b = 5'b00001; #10; // 31 * 1
48         $finish;
49     end
50 endmodule

```

OUTPUT REGISTER

```

Ln#
1  module OutputRegister (
2      input clk,
3      input reset,
4      input [9:0] data_in,
5      output reg [9:0] data_out
6  );
7      always @(posedge clk or posedge reset) begin
8          if (reset)
9              data_out <= 10'b0;
10         else
11             data_out <= data_in;
12     end
13 endmodule

```

TEST BENCH CODE

```

16
17
18  module tb_OutputRegister;
19      reg clk, reset;
20      reg [9:0] data_in;
21      wire [9:0] data_out;
22
23      OutputRegister dut (.clk(clk), .reset(reset), .data_in(data_in), .data_out(data_out));
24
25      initial begin
26          clk = 0;
27          forever #5 clk = ~clk;
28      end
29
30      initial begin
31          reset = 1; data_in = 10'b0; #10;
32          reset = 0; data_in = 10'b1010101010; #10;
33          data_in = 10'b1111000000; #10;
34          $finish;
35      end
36  endmodule

```

ALU VERILOG CODE:

```
// Full Adder
module FullAdder (
    input a, b, cin,
    output sum, cout
);
    wire axorb, aandb, axorbandcin;
    xor (axorb, a, b);
    xor (sum, axorb, cin);
    and (aandb, a, b);
    and (axorbandcin, axorb, cin);
    or (cout, aandb, axorbandcin);
endmodule

// 5-bit Adder
module Adder5 (
    input [4:0] a, b,
    output [4:0] sum
);
    wire [4:0] c;
    FullAdder fa0 (a[0], b[0], 0, sum[0], c[0]);
    FullAdder fa1 (a[1], b[1], c[0], sum[1], c[1]);
    FullAdder fa2 (a[2], b[2], c[1], sum[2], c[2]);
    FullAdder fa3 (a[3], b[3], c[2], sum[3], c[3]);
    FullAdder fa4 (a[4], b[4], c[3], sum[4], c[4]); // Fixed: connected cout
endmodule

// 10-bit Adder for Multiplier
module Adder10 (
    input [9:0] a, b,
    output [9:0] sum
);
    wire [9:0] c;
    FullAdder fa0 (a[0], b[0], 0, sum[0], c[0]);
    FullAdder fa1 (a[1], b[1], c[0], sum[1], c[1]);
    FullAdder fa2 (a[2], b[2], c[1], sum[2], c[2]);
```

```

FullAdder fa3 (a[3], b[3], c[2], sum[3], c[3]);
FullAdder fa4 (a[4], b[4], c[3], sum[4], c[4]);
FullAdder fa5 (a[5], b[5], c[4], sum[5], c[5]);
FullAdder fa6 (a[6], b[6], c[5], sum[6], c[6]);
FullAdder fa7 (a[7], b[7], c[6], sum[7], c[7]);
FullAdder fa8 (a[8], b[8], c[7], sum[8], c[8]);
FullAdder fa9 (a[9], b[9], c[8], sum[9], c[9]); // Fixed
endmodule

// 5-bit Subtractor
module Subtractor5 (
    input [4:0] a, b,
    output [4:0] diff
);
    wire [4:0] b_comp, b_twos;
    wire [4:0] one = 5'b00001;
    not (b_comp[0], b[0]);
    not (b_comp[1], b[1]);
    not (b_comp[2], b[2]);
    not (b_comp[3], b[3]);
    not (b_comp[4], b[4]);
    Adder5 add1 (.a(b_comp), .b(one), .sum(b_twos));
    Adder5 add2 (.a(a), .b(b_twos), .sum(diff));
endmodule

// 5-bit Multiplier
module Multiplier5 (
    input [4:0] a, b,
    output [9:0] product
);
    wire [9:0] p0, p1, p2, p3, p4, s1, s2, s3;

    assign p0 = b[0] ? {5'b00000, a} : 10'b0;
    assign p1 = b[1] ? ({5'b00000, a} << 1) : 10'b0;
    assign p2 = b[2] ? ({5'b00000, a} << 2) : 10'b0;
    assign p3 = b[3] ? ({5'b00000, a} << 3) : 10'b0;

```

```

    assign p4 = b[4] ? ({5'b00000, a} << 4) : 10'b0;

    Adder10 a1 (p0, p1, s1);
    Adder10 a2 (s1, p2, s2);
    Adder10 a3 (s2, p3, s3);
    Adder10 a4 (s3, p4, product);
endmodule
// Input Register (5-bit)
module InputRegister (
    input clk, reset,
    input [4:0] data_in,
    output reg [4:0] data_out
);
    always @(posedge clk or posedge reset)
        if (reset) data_out <= 5'b0;
        else      data_out <= data_in;
endmodule
// Output Register (10-bit)
module OutputRegister (
    input clk, reset,
    input [9:0] data_in,
    output reg [9:0] data_out
);
    always @(posedge clk or posedge reset)
        if (reset) data_out <= 10'b0;
        else      data_out <= data_in;
endmodule
// Top-Level ALU
module ALU_5bit (
    input clk, reset,
    input [4:0] A_in, B_in,
    input [1:0] sel, // 00 = add, 01 = sub, 10 = mul
    output [9:0] ALU_out
);

```

```

wire [4:0] A, B, sum, diff;
wire [9:0] prod;
reg [9:0] result;
InputRegister inA (.clk(clk), .reset(reset), .data_in(A_in), .data_out(A));
InputRegister inB (.clk(clk), .reset(reset), .data_in(B_in), .data_out(B));
Adder5 add (.a(A), .b(B), .sum(sum));
Subtractor5 sub (.a(A), .b(B), .diff(diff));
Multiplier5 mul (.a(A), .b(B), .product(prod));
always @(*) begin
    case (sel)
        2'b00: result = {5'b00000, sum};
        2'b01: result = {5'b00000, diff};
        2'b10: result = prod;
        default: result = 10'b0;
    endcase
end
OutputRegister outReg (.clk(clk), .reset(reset), .data_in(result),
.data_out(ALU_out));
endmodule

```

TEST BENCH CODE:

```

module tb_ALU_5bit;
    reg clk, reset;
    reg [4:0] A_in, B_in;
    reg [1:0] sel;
    wire [9:0] ALU_out;

    ALU_5bit dut (
        .clk(clk),
        .reset(reset),
        .A_in(A_in),
        .B_in(B_in),
        .sel(sel),
        .ALU_out(ALU_out)
    )

```

```

);

// Clock generation
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// Test cases
initial begin
    $display("Time\tA\tB\tSel\tResult");
    $monitor("%0t\t%b\t%b\t%b\t%b", $time, A_in, B_in, sel, ALU_out);

    reset = 1; A_in = 0; B_in = 0; sel = 0; #10;
    reset = 0; #10;

    // ADD: 5 + 3 = 8
    A_in = 5'b00101; B_in = 5'b00011; sel = 2'b00; #20;

    // SUB: 12 - 5 = 7
    A_in = 5'b01100; B_in = 5'b00101; sel = 2'b01; #20;

    // MUL: 3 * 5 = 15
    A_in = 5'b00011; B_in = 5'b00101; sel = 2'b10; #30;

    // MUL: 31 * 31 = 961
    A_in = 5'b11111; B_in = 5'b11111; sel = 2'b10; #30;

    $finish;
end
endmodule

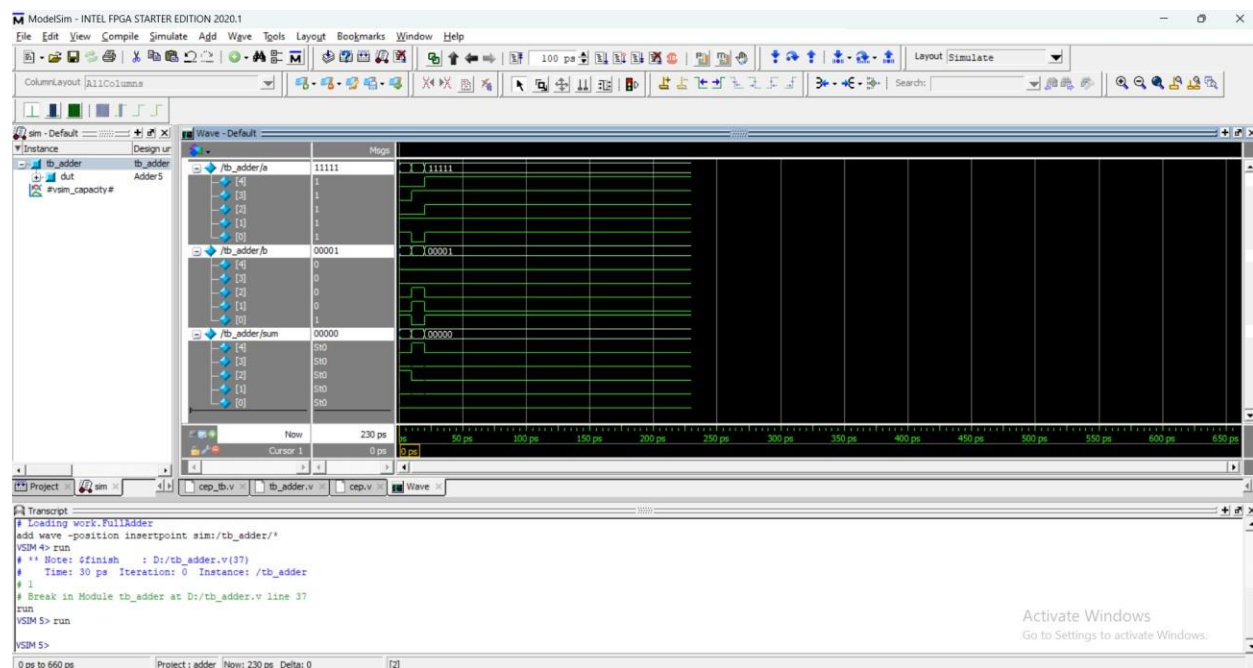
```

CONTROL TABLE:

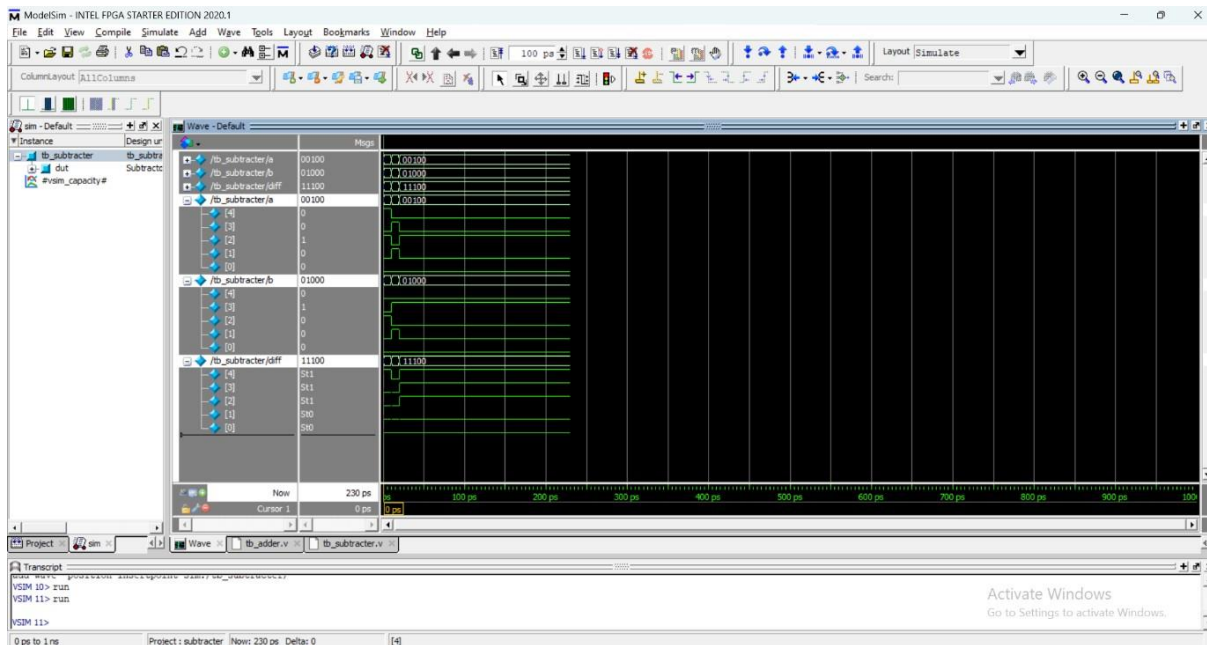
op_code	Operation	Module Used	Output Width
00	Addition	Adder5	5-bits(zero-padded to 10)
01	Subtraction	Subtractor5	5-bits(zero-padded to 10)
10	Multiplication	Multiplier5	10-bits
11	N/A	Default zero	10-bits

TRUTH TABLE:

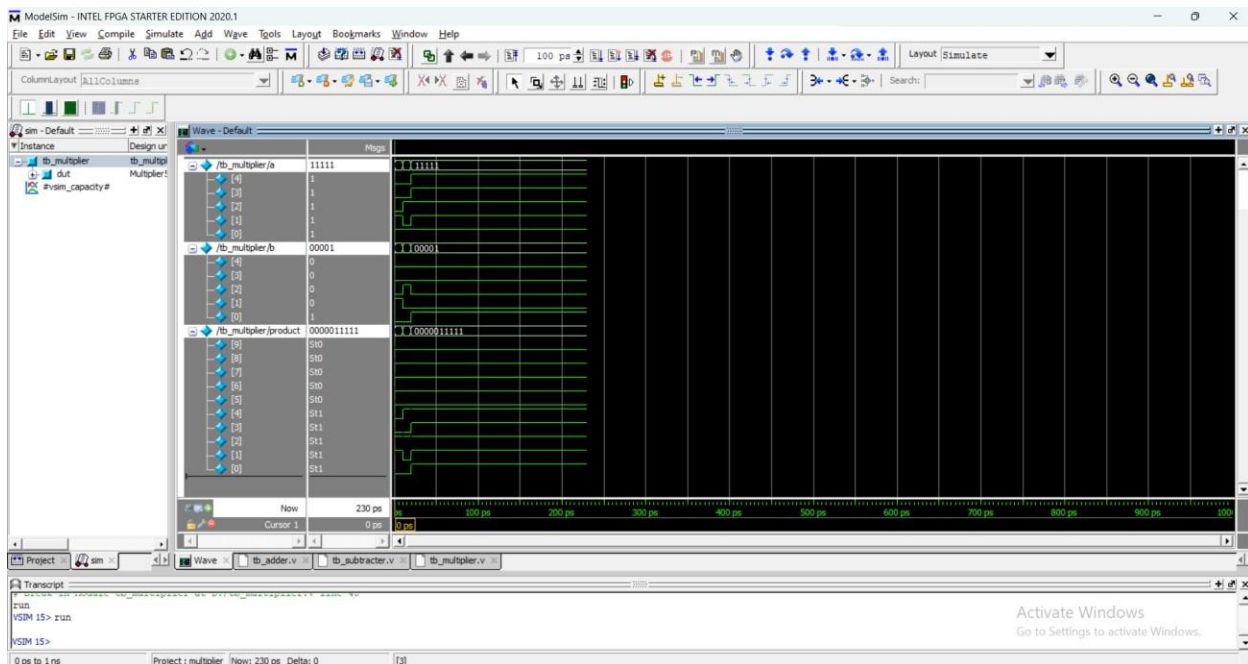
(b)a	(b)b	op	operation	result(b)
00101	00011	00	Addition	0000001000
00100	00000	00	Addition	0000000100
00101	00011	01	Subtraction	0000000010
00100	00000	01	Subtraction	0000000100
00101	00011	10	Multiplication	0000001111
00100	00010	10	Multiplication	0000001000
Any	Any	11	Invalid/Reset	0

SIMULATION RESULTS:**ADDER:**

SUBTRACTOR:



MULTIPLIER:



README FILE:

Readme file is also attached with Digital Logic Design CEP pdf in txt format named "README.txt"

PROBLEMS FACED & SOLUTIONS:

Problem	Solution
Verilog doesn't allow module instantiation inside functions	Rewrote multiplication as unrolled chained adder logic outside functions
Waveform not showing in ModelSim	Ensured simulation ran long enough and used add wave * before run
Multiplication overflow	Used 10-bit result register to capture full product
Registered output delay	Understood output changes one clock after operation due to registers

CONCLUSION:

In this project, a 5-bit Arithmetic Logic Unit (ALU) was successfully designed using **gate-level Verilog**, following strict digital logic principles. The ALU performs **addition, subtraction, and multiplication** using only basic logic gates and structural modeling. Multiplication was implemented using a shift-and-add method without using behavioral operators like `*`, adhering strictly to the project constraints. All inputs and outputs were **registered**, ensuring synchronous data flow with the clock. A **Moore Finite State Machine (FSM)** was used to control the sequential operation of the ALU, improving reliability and timing predictability.

This project deepened understanding of digital system design, FSM implementation, gate-level hardware modeling, and the complete flow from RTL coding to simulation and synthesis.

RESULTS:

- **Functionality Verified:** All operations (ADD, SUB, MUL) worked correctly in simulation.
- **Simulation Tool:** ModelSim
- **Test Vectors Applied:** Multiple combinations of 5-bit inputs (A, B) were tested with control `sel` = 00, 01, 10.
- **Waveform Analysis:** All outputs matched expected values with proper register delays.
- **Operation Widths:**
 - ADD, SUB → 5-bit outputs
 - MUL → 10-bit output to accommodate full product
- **Maximum Case Tested:**
 - Input A = 31 (11111), B = 31 (11111)
 - Multiplication Result = 961 (1111000001), confirmed in waveform